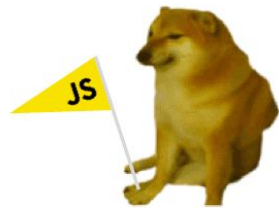




De strikte maar krachtige superset
van JavaScript



Agenda



1. TypeScript compiler
2. TypeScript Basis Types
3. TypeScript Custom Types
4. Geavanceerde Technieken
5. TypeScript met NodeJS

Context & Inleiding

TypeScript

- *Superset* van JavaScript door Microsoft
 -  Docs: <https://www.TypeScriptlang.org/docs>
 -  Probeer uit: <https://www.typescriptlang.org/play>
- **Programmeertaal met strikte types**
 - Voegt **types** en **object geörienteerde** uitbreidingen toe aan JS
- Vangt **fouten eerder** op
 - In JavaScript worden fouten gedetecteerd tijdens het uitvoeren
 - In TypeScript worden fouten gedetecteerd bij het compileren (door de TypeScript compiler tsc) of tijdens het schrijven (door de IDE editor)
- De meeste populaire JS packages bieden getypeerde definities aan ter ondersteuning voor TypeScript

TypeScript

addFiveJs.js

```
function addFive(num) {  
    return num + 5;  
}  
console.log(addFive("2")); // 25
```

Er wordt een 5 toegevoegd aan een String.

Javascript doet alles wat je vraagt, maar wil je dit ook echt?

addFiveTs.ts

```
function addFive(num : number){  
    return num + 5  
}  
console.log(addFive("2")) // Error
```

*Je kan geen String toekennen aan number
-> compilatie fout*



Voorbeeld

<https://gitlab.com/kdg-ti/web-technology-2/m03-TypeScript/examples>

TS2345: Argument of type 'string' is not assignable to parameter of type 'number'. :

[Change addFive\(\) signature](#) Alt+Shift+Enter [More actions...](#) Alt+Enter

tsc

- Je browser begrijpt alleen JavaScript
 - node.js >=v24 kan eenvoudige ts bestanden rechtstreeks draaien
- Ts moet (meestal) getranspileerd worden.
 - “tsc” → TypeScript compiler
 - JavaScript als output



tsc

- Installeer TypeScript in je project (of globaal)

```
$ pnpm i -D typescript
```

- Hernoem je `index.js` naar `index.ts`
- Transpileer je `index.ts` code met **tsc**. Dit genereert het overeenkomstige `index.js` bestand.

```
$ pnpm tsc index.ts
```

TypeScript compiler

TypeScript bestand

tsconfig.json

tsconfig.json bestand in de top directory van je project:

- specificeert tsc opties en parameters zodat je ze niet elke keer moet intikken
- WebStorm pikt deze configuratie op en zal deze gebruiken om statisch types te gaan checken.

Voorbeeld tsconfig.json (browser)

- Verzameling aanbevolen configuratie bestanden:
<https://github.com/tsconfig/bases>
 - a. Installeer de **recommended** basis configuratie in de devDependencies:
\$ pnpm i -D @tsconfig/recommended
 - b. Aanbevolen configuratiebestand voor browser ts:

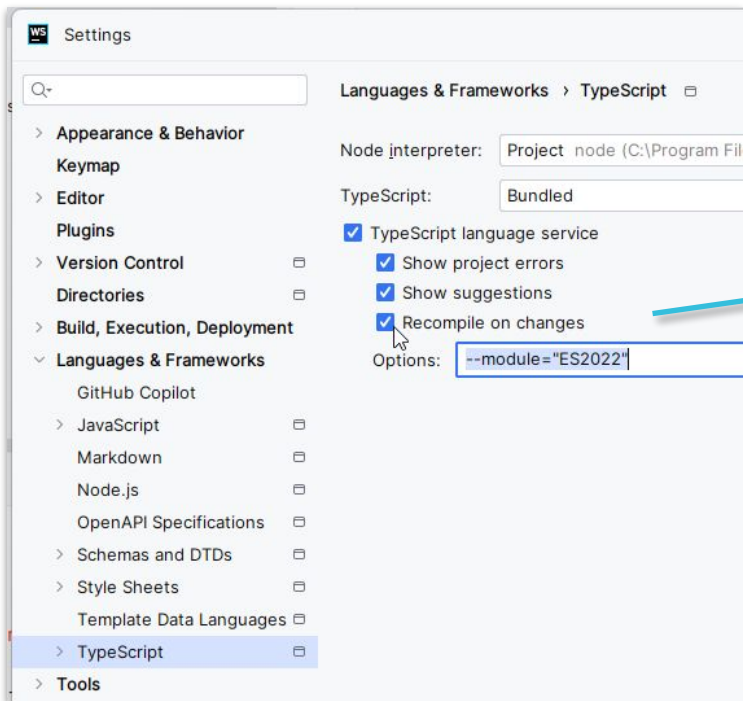
tsconfig.json

```
{
  "extends": "@tsconfig/recommended/tsconfig.json",
  "compilerOptions": {
    "module": "es2022",
    "target": "es2024",
    "moduleResolution": "node",
    "lib": ["ESNext", "DOM", "DOM.Iterable"]}
}
```



Je kan WebStorm ook (automatisch) laten transpileren...

File | new Projects Setup | Settings for new projects

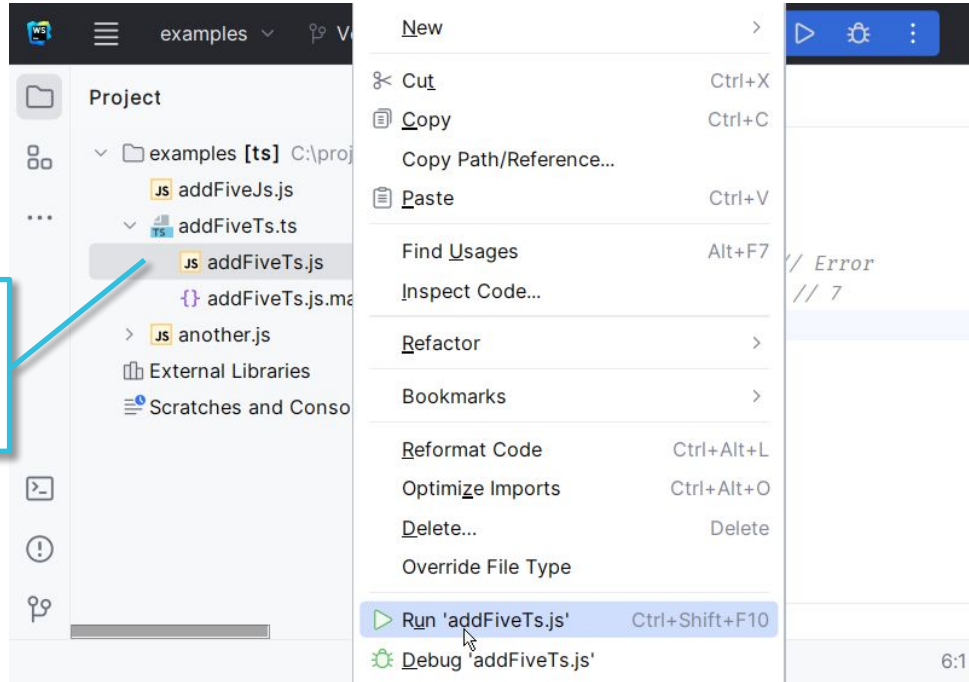


webstorm compileert automatisch na aanpassing

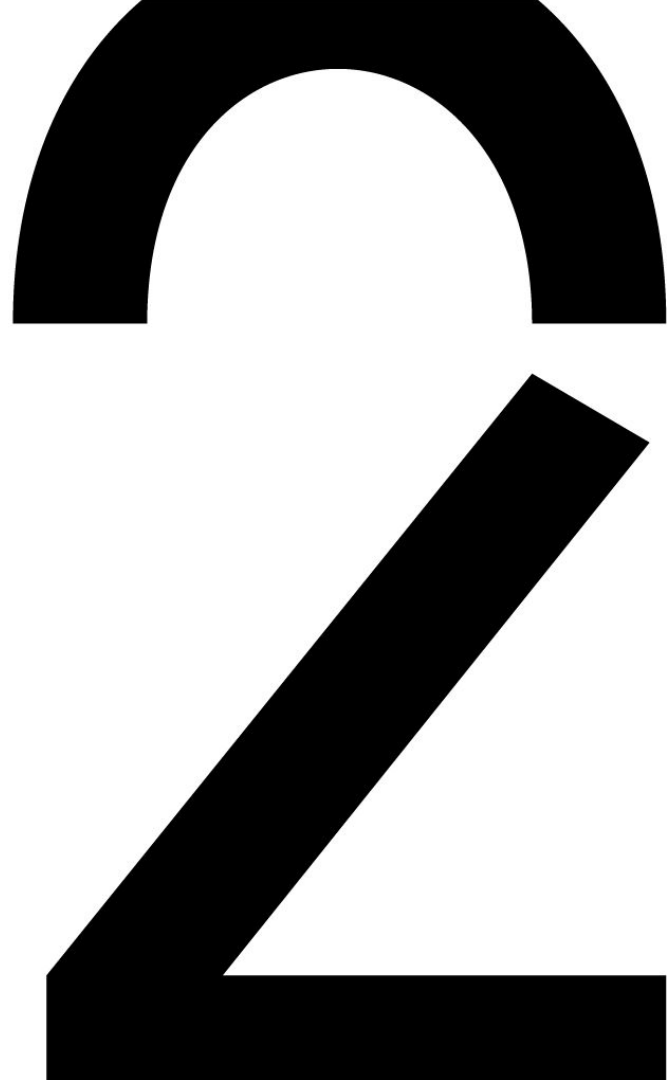


...en dan het gegenereerde .js bestand runnen:

*.js bestand wordt automatisch
gegenereerd
Right click + run*

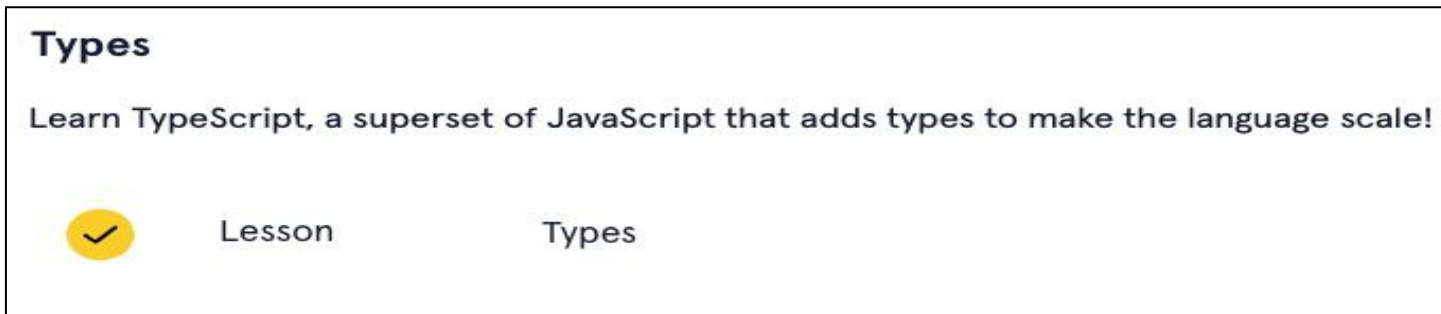


Basis Types



Codecademy learn TypeScript

Optioneel kan je in de [interactieve codecademy Learn TypeScript cursus](#) de les over types doorlopen. Je kan je kennis testen en uitproberen.



En nog een goede cursus: [learntypescript.dev](#)

Primitieve types

Ts heeft 3 veel gebruikte Primitieve Types

- boolean
- number (geheel of komma getal)
- string

```
const isKlaar : boolean = false;  
const lijnen : number = 42;  
const naam : string = "Peter";
```

Type Inference / (afleiding)

TypeScript probeert het type voor een variabele af te leiden als er geen type-annotatie is

```
let score = 10;
```

Welk type krijgt **score**?

```
let dateOfBirth = new Date(1990, 4, 7);
```

Welk type krijgt **dateOfBirth**?

Er werd afgeleid dat het type van **score** een **number** is.

TypeScript heeft afgeleid dat het type van **dateOfBirth** een **Date** is

=> **Date** is geen JavaScript type, het is dus een TypeScript-specifiek type.

Types in functies

```
function f1 (i : number): number {  
    return i * i;  
};
```

//Zelfde functie als lambda

```
const f2 = (i: number): number => i * i;
```

*Return type kan meestal door TypeScript afgeleid worden en hoef je dan niet mee te geven.
Parameter types moet je wel meegeven*

```
export function randomNumbers(size) : number[] {  
    return Array(size).map (i => M  
};
```

TS7006: Parameter `size` implicitly has an `any` type.

Infer parameter types from usage `Alt+Shift+Enter` More actions... `Alt+Enter`

`size: any`

WebStorm kan je eventueel helpen parameter types af te leiden (ALT+SHIFT+ENTER)

Types in functies

// void is het return type voor functies die niets terug geven

```
function groteBoodschap(): void {  
    console.log("Ik ben een vervelende bericht!");  
}
```

// Gebruik ? voor optionele parameters

```
function groet(naam? : string) {  
    console.log(Hallo, ${naam || "geen naam"}!);  
}
```

Basis Types: arrays

```
// Voor collecties zijn er "typed arrays" type[]  
const lijst: number[] = [1, 2, 3];  
  
// Typescript ondersteunt ook <Generics>  
const lijstAlternatief: Array<number> = [1, 2, 3];
```

```
// Definieer een tweedimensionale array met getallen  
let matrix: number[][] = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
];  
// Toegang tot elementen in de array  
console.log(matrix[0][1]); // Geeft 2 weer
```

Basistypes: Tuples

Heeft de syntax van een array, maar lijkt op een object zonder attributnamen: vast aantal elementen én types.

```
// Definieer een tuple met een string, een number en een boolean  
let myTuple: [string, number, boolean] = ['Hello', 42, true];  
  
// met 3 numbers  
let numbersTuple: [number, number, number] = [1, 2, 3];
```

Custom Types

De ware kracht van TypeScript



Object types

```
const tomScore = {  
  name: "Tom",  
  score: 70  
};
```

Wat is het type van tomScore?

Het type van tomScore wordt afgeleid (of "**Inferred**") als:

```
const tomScore : {name: string, score: number} = {  
  name  
  score  
};  
console.log(tomScore);
```

ts/object-types.ts

interfaces en types

- Er zijn 2 "smaakjes" interface of type alias

interface

```
interface Dier {  
    naam: string;  
}  
  
interface Kat extends Dier {  
    huiskat?: boolean;  
    miauw(): void;  
}  
  
const kat : Kat = {  
    naam: "Fluffy",  
    huiskat: true,  
    miauw: () => {  
        console.log('Miauw!')  
    }  
}
```

Aanbevolen manier

type

```
type Dier = {  
    naam: string;  
}  
  
type Kat = Dier & { huiskat?: boolean,  
    miauw: () => void }  
  
const kat: Kat = {  
    naam: "Fluffy",  
    huiskat?: true,  
    miauw: () => {  
        console.log('Miauw!')  
    }  
}
```

'&' operator maakt een
intersection type
(AND)

? geeft een optioneel
attribuut aan



<https://stackoverflow.com/questions/37233735/interfaces-vs-types-in-TypeScript/65948871#65948871>

Functies

- **Interfaces** en **Types** kunnen beide ook *functies* beschrijven

```
interface FuncInter {(bron: string, sub: string): boolean;}  
type FuncType = (bron: string, sub: string) => boolean;
```

```
let mySearch: FuncInter ;  
// of  
let mySearch: FuncType ;
```

```
mySearch = function (src: string, sub: string) {  
    return src.search(sub) !== -1;  
}
```

```
console.log(mySearch("interfaces en types","terf")); //true
```

Lambda syntax

Functies

functionParams.t

S

```
let currentDisplay = 0;  
let currentKey:string;
```

Een functie als
parameter

```
function process(entered: number,  
  operation:(first: number, second: number)⇒ number) {  
  const result = operation(currentDisplay,entered);  
  console.log(  
    `${currentDisplay} ${currentKey} ${entered} = ${result}`);  
  currentDisplay = result;  
}
```

```
function calculate(operation: string,entered: number) {  
  currentKey=operation ;  
  switch (currentKey) {  
    case "+":  
      process(entered, (a, b) ⇒ a + b);  
      break;  
    case "-": //weggelaten...  
  }  
}
```

```
0 + 5 = 5  
5 * 9 = 45  
45 - 11 = 34  
34 / 4 = 8.5  
8.5 ^ 2 = 72.25
```


Union Types (OR)

- nieuwe types bouwen uit bestaande types
- Gebruikt operator `|` (OR)

```
// combineer types.  
// id kan een string of een number zijn  
function printId(id: number | string ) {  
    console.log("Your ID is: " + id);  
}  
// OK  
printId(101);  
// OK  
printId("202");  
// Compile Error  
printId({myID: 22342});
```

*object is niet toekenbaar aan
number -> compile error*

Union Types

- Union type kan ook met literals

```
type WindowStates = "open" | "closed" | "minimized";
```



<https://learnTypeScript.dev/04/l10-quiz>

enums

- Alternatief voor union types met literals

```
enum Direction {  
    North="NORTH",  
    South="SOUTH",  
    East="EAST",  
    West="WEST"  
}  
  
let goWest = Direction.West;  
  
console.log(Direction); // [ 'NORTH', 'SOUTH', 'EAST', 'WEST' ]  
console.log(goWest); // WEST  
// goWest="EAST"; // compiler error  
goWest = "EAST" as Direction; // OK
```

Partial <T>

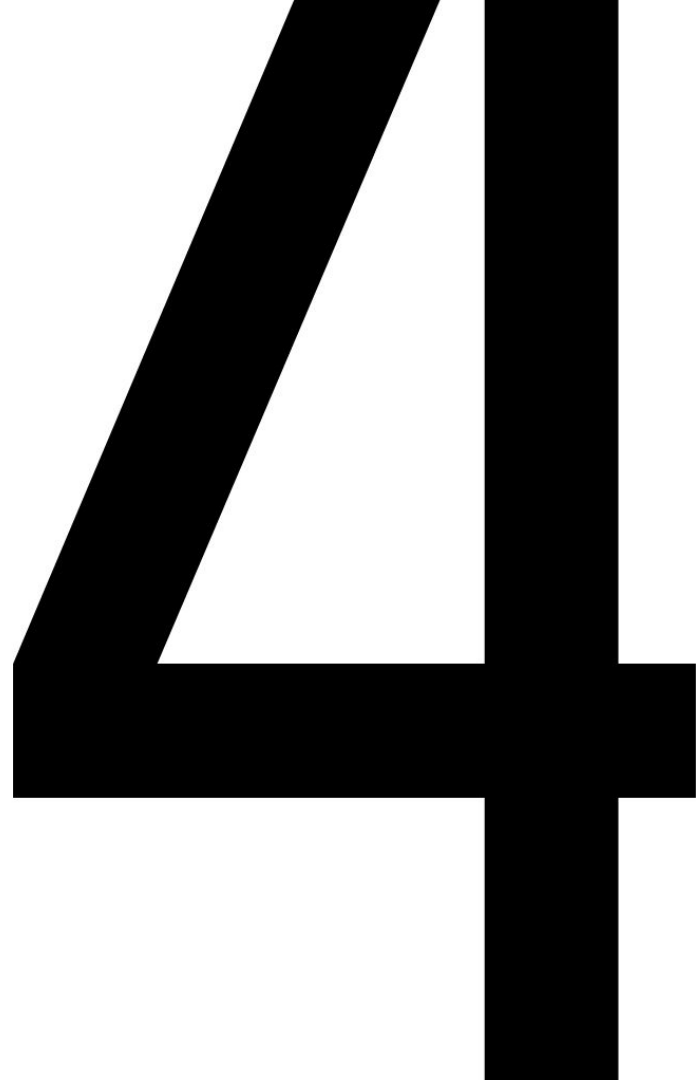
- Variant van een type waar alle attributen optioneel zijn.
- Handig om gedeeltelijk opgevulde objecten door te geven
 - Tegenhanger Required <T>: alle attributen verplicht

partial.ts

```
interface Todo {  
  task: string;  
  description: string;  
}  
  
const urgent = {  
  task: "organize desk",  
  description: "clear  
clutter",  
};
```

```
function updateTodo(todo: Todo,  
  fieldsToUpdate: Partial<Todo>) {  
  return { ...todo, ...fieldsToUpdate  
};  
}  
  
console.log( updateTodo(todo1,  
  { description: "throw out trash"}  
);  
// { title: 'organize desk',  
//   description: 'throw out trash' }
```

Geavanceerde technieken



Shape checking

TypeScript checkt of de waarde die je toekent ***minstens*** de **vorm** (properties en methoden) van het type van de variabele heeft.

```
interface Point { x: number; y: number;}  
function logPoint(p: Point) { console.log(`${p.x}, ${p.y}`);}  
  
const ThreeD = { x: 33, y: 3, z: 30};  
logPoint(ThreeD); // meer properties is OK: logs "12, 26"  
  
const OneD = { x: 33};  
logPoint(OneD); // Compile Error: property y missing
```

Type checking

Om veilig attributen/methodes te gebruiken, kan je zelf ook op een type checken met `typeof`

```
if (typeof value === "string") {  
    console.log(value.length);  
}
```

Op objecten kan je ook checken met `instanceof`

```
if ( value instanceof Error) {  
    throw(value);  
}
```

any type (I don't care)

- Schakelt type controle uit
- toegang tot alle eigenschappen ervan
 - op hun beurt opnieuw van het type 'any'



```
let nietZeker: any = 4;  
nietZeker = "misschien een string";  
nietZeker = false; // Toch een boolean
```

```
let ding: any = {x: 0};  
ding.foo(); // RUNTIME ERROR  
ding();     // RUNTIME ERROR  
ding.bar = 100;  
ding = "hello";  
const n: number = ding;
```

Je kan doen wat je wil, de tsc compiler zal niet klagen

unknown type (I don't know)

- testen voor gebruik!
- Indien je type niet weet: verkies `unknown` boven `any`!

```
let nietZeker: unknown = 4;  
nietZeker = "misschien een string";  
nietZeker = false; // Toch een boolean
```

Je kan toekennen wat je wil

```
nietZeker.toUpperCase(); // ✗ fout bij compile
```

COMPILE ERROR

👉 Wat betekent dit?

- **unknown** mag elke waarde krijgen (net als `any`).
- **Maar** TypeScript behandelt het **altijd als "ik weet niet wat dit is"**.
- Je kunt er **niet zomaar dingen mee doen** zonder eerst te checken.

unknown type (I don't know)

- testen voor gebruik!
- Indien je type niet weet: verkies `unknown` boven `any`!

```
let ding: unknown = {x: 0};
```

eerst `checken` dan gebruiken

```
if (typeof ding === "function") {  
    ding();  
}
```

```
// ding.bar = 100; // COMPILE error: Property 'bar' does not exist  
ding = 5;
```

*met `if` geen **COMPILE ERROR***

```
if (typeof ding === "number") {  
    const n: number = ding;  
} else {  
    console.log("ding is geen number of functie");  
}
```

Type assertion

```
// Selecteer een element en vertel TypeScript dat het een HTMLInputElement is  
const inputElement = document.getElementById("myInput") as HTMLInputElement;  
  
// Nu kun je de specifieke methoden en eigenschappen van HTMLInputElement gebruiken  
inputElement.value = "Hello, world!";  
inputElement.disabled = true;  
  
// Een andere manier om een type assertion te doen, is met behulp van de angle-bracket notation  
const anotherInputElement = <HTMLInputElement>document.getElementById("anotherInput");
```

Let op voor fouten! Dit verzwakt type checking. Als het opgehaalde element bv een <h1> is zal `inputElement.value` at runtime falen

Type assertion non-null assertion operator (!)

```
// Selecteer het formulier element en vertel TypeScript dat het bestaat  
const formElement = document.getElementById("myForm")!;  
  
// Voeg een event listener toe aan het formulier  
formElement.addEventListener("submit", (event) => {  
  // Voorkom het standaardgedrag van het formulier  
  event.preventDefault();  
  
  // Voer je eigen logica uit hier...  
});
```

! wordt geplaatst achter een variabele of uitdrukking waarvan we TypeScript willen vertellen dat die niet null of undefined is.

Als de !-uitdrukking well null is krijg je een runtime fout

Types en modules

- Over het algemeen definieer je je types in een afzonderlijke module

Point.ts

```
export interface Point { x: number; y: number;}
```

- Typescript heeft de import syntax hiervoor uitgebreid

```
import type {Point} from "./Point";
```

Een indicatie voor TypeScript dat deze import niet behouden moet worden bij transpilatie

Commandline TypeScript met Node.js

Aanbevelen tsconfig.json

- Voor server side TypeScript in node.js

```
$ pnpm i -D @tsconfig/node-lts
```

tsconfig.json

```
{
  "extends": "@tsconfig/node-lts/tsconfig.json",
  "compilerOptions": {
    "module": "node20"
  }
}
```

tsx

tsx is een wrapper rond node.js die

- TypeScript begrijpt: je hoeft niet eerst te compileren
- ES Modules begrijpt

```
$ pnpm i typescript tsx -D
```

- in package.json vervang je in de scripts node door tsx. Voorbeeld:

```
"scripts": {  
  "start": "tsx index.ts"  
}
```


TypeScript met NodeJS

Om TypeScript te laten werken met NodeJS en zijn libraries (e.g. de 'fs' module), kunnen we de **binding** van NodeJS installeren in ons project.

Wat zijn TypeScript Bindings ?

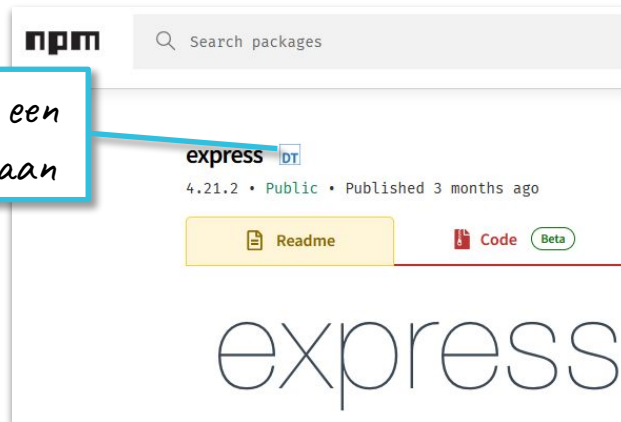
- Bindings voegen types en interfaces toe aan de functies in een bibliotheek of module die in Javascript geschreven zijn
- In xxx.d.ts bestanden: deze bevatten enkel declaraties, geen uitvoerbare code
- Op die manier kan je toch *typechecking* doen in je ts code die js libraries gebruikt

Hoe installeer ik een Binding?

- Zoek de TypeScript-binding: De naam van de binding begint meestal met "@types/" gevolgd door de naam van de JavaScript-bibliotheek of -module.
- Installeer de TypeScript-binding voor nodeJS:

```
$ pnpm i -D @types/node
```
- Je vindt een verzameling in het [DefinitelyTyped repository](#).

Op de npm pagina vind je ook een link naar de types als ze bestaan



Hoe installeer ik een Binding?

- Je kan nu de fs module importeren zonder foutmeldingen:

```
import * as fs from "node:fs";
```

package.json

```
{  
  "name": "my project",  
  "type": "module",  
  "version": "1.0.0",  
  "devDependencies": {  
    "@tsconfig/node-lts": "^20.1.1",  
    "@types/node": "^20.11.19",  
    "TypeScript": "^5.3.3",  
    "tsx": "^4.7.1"  
  },  
  "scripts": {  
    "build": "tsc",  
    "start": "tsx"  
  }  
}
```



1. TypeScript compiler
2. TypeScript Basis Types
3. TypeScript Custom Types
4. Geavanceerde Technieken
5. TypeScript met NodeJS



Online

- [TypeScript for JavaScript Developers](#)
- [TypeScript Handbook](#)
- <https://learnTypeScript.dev/>
- [!\[\]\(2824aab9645d9fab95bae27ff6828dab_img.jpg\) "The definitive guide to TypeScript and possibly the best TypeScript book" !\[\]\(0fbf3ad74a6c8dc44ba9ea17fc2aca5e_img.jpg\).](#)
(Free and Open Source)
- [codecademy TypeScript cursus](#)

